

Assembleia 04 — Implementação: Personagem e Orçamento

Continuação direta da Assembleia 03 (docs/ASSEMBLEIA-03-PERSONAGEM-E-ORCAMENTO.md), que já decidiu **o quê** fazer a seguir (personagem primeiro, orçamento como bloqueio antes de mais testers) mas deixou **como** fazer em aberto, de propósito ("fica pra implementação real"). Esta assembleia resolve o desenho técnico dos dois, com o método completo.

1. Baseline (munição pro debate, não proposta fechada)

Números pesquisados agora, não de memória (console.groq.com/docs/rate-limits, 31/08/2026, modelo openai/gpt-oss-120b, tier gratuito, **por organização inteira**, não por jogador):

Limite	Valor
RPM (requisições/minuto)	30
RPD (requisições/dia)	1.000
TPM (tokens/minuto)	8.000
TPD (tokens/dia)	200.000

Estimativa honesta recalculada (não medida de verdade — hoje não existe nenhum registro de uso real de token no código, só o pass-through da resposta da Groq): trimHistory cobre intro + 10 entradas recentes, então o prompt de entrada estabiliza em torno de ~2.150 tokens (system prompt ~600 + histórico ~1.500) depois dos primeiros turnos, mais max_completion_tokens: 700 de saída — **~2.850 tokens por turno**. Uma campanha de 40 rodadas chegaria perto de **~114.000 tokens**, quase o dobro da estimativa registrada anteriormente em CLAUDE.md (que citava ~60k) — a diferença importa pro dimensionamento do mecanismo de proteção, e reforça por que medir de verdade (Seção 2, AI Master Engineer) é melhor que estimar de novo.

Duas frentes, ambas já aprovadas em princípio:

- **A — Criação de personagem:** a tabela characters já existe (supabase/schema.sql), com user_id/name/data jsonb, e está morta — zero referência em src/. Perguntas reais de desenho: em que momento do fluxo aparece; quais campos o jogador preenche; os 3 NPCs companheiros (Brennan/Sira/Korrin) seguem fixos (fora de escopo — a Assembleia 03 falou só de personagem do jogador).
- **B — Orçamento de token:** onde mora o contador agregado, como evita condição de corrida entre jogadores simultâneos, se usa dado real da própria resposta da Groq ou estimativa, e se isso muda o número atual do limite de demo (DEMO_LIMIT = 40).

2. Consulta individual aos agentes

Backend Engineer

Pra personagem: CRUD simples sobre a tabela characters já desenhada — não é escopo novo de dado, é ligar o que já existe. Pra orçamento: proponho uma tabela nova (groq_usage_daily, data + tokens usados) com incremento via **função RPC atômica no Postgres** (não ler-modificar-escrever em JavaScript) — evita duas requisições simultâneas lendo o mesmo valor antes de qualquer uma escrever.

Game System Designer

Personagem: nome + arquétipo — prefiro **3 ou 4 arquétipos pré-definidos** (cada um já com CRP/MNT/ALM calculados e balanceados) em vez de distribuição livre de pontos — mais rápido de entregar e sem risco de combinação desbalanceada logo na v1. HP/MP calculados pela fórmula já existente ($10 + \text{CRP}$, $4 + \text{ALM} \times 2$). Reafirmo: NPCs fora de escopo. Sobre orçamento, não é meu domínio, mas se o número de 40 rodadas mudar, isso afeta o ritmo narrativo dos arcos — quero ser consultado antes de qualquer novo número ser fixado, não descobrir depois.

Frontend Engineer

Personagem: um CharacterCreationGate novo, no mesmo padrão visual do LoginGate/SetPasswordGate, aparecendo quando não existe personagem salvo pro usuário.

Divergência real com o Game System Designer: acho arquétipos fixos uma regressão do que foi pedido desde a Assembleia 01 ("criação de personagem" como item não-negociável, implicitamente sugerindo liberdade de escolha) — prefiro **distribuição livre de pontos**, mesmo custando mais tela. Sobre orçamento: se o jogador vai ver algum aviso de cota chegando perto do fim, isso é meu — preciso saber se o desenho prevê aviso visual ou só bloqueio silencioso no backend antes de desenhar a tela.

AI Master Engineer

O orçamento é o item mais urgente tecnicamente, e quero resolver com **dado real, não estimativa**: a resposta da Groq (compatível com o formato OpenAI) deveria trazer `usage.total_tokens` — isso precisa ser **confirmado na implementação real**, não assumido aqui. Se existir, `api/master.js` grava esse número real na tabela agregada depois de cada chamada bem-sucedida, e verifica o total do dia **antes** de cada chamada nova, com margem de segurança — se perto do teto, devolve o mesmo `quota_exceeded` já tratado no frontend, sem gastar nem 1 token a mais. **Meu voto tem uma condição:** não sou a favor de simplesmente baixar o número de 40 rodadas como solução — isso é tampar sem saber se resolve; o mecanismo de verdade é o que protege, o número é cosmético por cima dele.

Infra Engineer

Sem custo extra pra uma tabela nova no Supabase, mesmo tier free. Um UPDATE/INSERT simples de contador não é motivo de preocupação de latência.

Code QA Engineer

Personagem: RLS já existe em characters, isolamento coberto, sem necessidade de revisão adicional aqui. Orçamento: **reforço o ponto do Backend Engineer com peso de segurança, não só de arquitetura** — se o contador for lido e incrementado sem atomicidade real, dois jogadores terminando um turno ao mesmo tempo podem os dois passar pela checagem antes de qualquer incremento ser gravado, furando o teto exatamente no momento em que mais protege. Função RPC atômica não é preferência de estilo, é requisito de segurança real aqui.

Narrative Writer

Personagem: o texto de onboarding do fluxo de criação mantém o tom "sábio, direto" do resto do jogo — não é um formulário burocrático, é parte da entrada na história. Sem posição forte sobre orçamento.

Test Engineer

Quero testar criação de personagem como parte do fluxo encadeado real (criar → jogar alguns turnos → fechar e reabrir → personagem continua). Sobre orçamento: **divirjo na urgência** — prefiro uma medida paliativa imediata (baixar o número de demo já, esta semana) enquanto o mecanismo de verdade não está pronto, porque não sei quanto tempo a implementação completa vai levar, e cada dia sem proteção nenhuma é risco real pra qualquer tester que já possa aparecer. Pergunta técnica separada: preciso de um jeito de simular/forçar o teto em ambiente de teste sem gastar 200k tokens reais pra validar o bloqueio.

Realtime Multiplayer Engineer

Sem posição forte nos dois temas — não envolvem sala. Só uma nota pro futuro: se/quando sala voltar, o contador de orçamento precisa virar por sala ou global, nunca por jogador individual — registrando aqui pra não repetir o mesmo tipo de esquecimento que gerou a Retrospectiva 01.

3. Síntese — convergência e divergência

Convergência forte: personagem liga a tabela já existente, sem escopo novo de dado; NPCs ficam fixos; orçamento precisa de mecanismo atômico no banco (Backend + Code QA, sem discordância); dado real da Groq é preferível a estimativa (ninguém defendeu o contrário).

Três divergências reais, viram eixo de variação dos finalistas:

- 1 **Arquétipos fixos (Game System Designer) vs. distribuição livre de pontos (Frontend Engineer)** — tensão entre velocidade/balanceamento e liberdade de personalização.
 - 2 **Ordem: contador primeiro (AI Master Engineer, Code QA) vs. personagem e contador em paralelo (maioria)** — quanto peso dar à urgência do orçamento vs. ao atraso já acumulado da criação de personagem.
 - 3 **Paliativo imediato — baixar o número de demo já (Test Engineer) — vs. esperar o mecanismo de verdade (AI Master Engineer rejeita isso explicitamente, chama de "tampar sem saber se resolve").**
-

4. Cinco finalistas

Finalista 1 — "Contador primeiro, personagem depois"

Resolve o mecanismo de proteção (com dado real da Groq) e só depois disso entra a criação de personagem. Mais lento pra destravar personagem, mais rápido pra proteger o orçamento.

Finalista 2 — "Personagem e contador em paralelo, arquétipos pré-definidos"

Os dois avançam ao mesmo tempo (não competem — times/partes diferentes do código); personagem usa arquétipos fixos pra ser rápido de entregar.

Finalista 3 — "Personagem e contador em paralelo, distribuição livre de pontos"

Igual ao Finalista 2, mas personagem com distribuição livre — mais trabalho de Frontend/Game System Designer, mais liberdade de jogador.

Finalista 4 — "Baixar o limite de demo já, como paliativo imediato"

Reduz DEMO_LIMIT de 40 pra um número mais seguro nesta semana, enquanto o contador de verdade (dado real da Groq) é construído em paralelo com personagem (arquétipos fixos).

Finalista 5 — "Mínimo absoluto: só personagem, orçamento fica manual"

Constrói só a criação de personagem agora; pro orçamento, em vez de mecanismo automático, reforça que o Tiago controla manualmente quantos convites manda por dia até o mecanismo de verdade ser priorizado depois. Mais rápido de entregar, não resolve o risco técnico real.

5. Votação

Agente	Voto	Justificativa / objeção
AI Master Engineer	#1	Orçamento é o item com prazo real; objeta a #5 — "é o mesmo padrão de deriva que gerou a Retrospectiva 01, só que de novo".
Code QA Engineer	#1	Concorda — risco de segurança/estabilidade real vem antes de feature nova; mesma objeção a #5.
Backend Engineer	#2	Paralelo é eficiente — personagem (schema já pronto) e a função RPC de orçamento não competem pelo mesmo tempo de trabalho.
Game System Designer	#2	Arquétipos, quer destravar personagem logo — já atrasado desde a Assembleia 01.
Narrative Writer	#2	Mesma razão do Game System Designer — conteúdo parado esperando personagem.
Frontend Engineer	#3 (dissenso)	Distribuição livre é mais fiel ao pedido original de "criação de personagem"; objeta à parte de arquétipos do #2, não ao paralelismo em si.

Infra Engineer	#2	Sem diferença de custo; segue o consenso do paralelismo.
Test Engineer	#4 (dissenso)	Quer o paliativo imediato — baixar o número já — porque a implementação completa pode levar dias e a proteção zero durante esse tempo é risco real.
Realtime Multiplayer Engineer	#2	Sem posição forte; segue o consenso.

Resultado: Finalista 2 vence 5 a 2 a 1 a 1 (Frontend Engineer dissente pra #3; Test Engineer dissente pra #4; AI Master Engineer e Code QA Engineer dissentem juntos pra #1).

6. Vencedor — com as mitigações da minoria incorporadas

Finalista 2, "Personagem e contador em paralelo, arquétipos pré-definidos", com três ajustes vindos dos dissensos que têm peso técnico real:

- 1 Do dissenso do AI Master Engineer/Code QA Engineer (#1)** — incorporado, não descartado: o desenvolvimento é paralelo, mas o **lançamento não é** — o contador usa dado real desde o primeiro commit, e **nenhum novo convite de teste sai até os dois estarem prontos e testados em produção**, exatamente como a Assembleia 03 já havia decidido. Paralelo no código, sequencial na exposição a usuários reais.
- 2 Do dissenso do Frontend Engineer (#3)** — parcialmente incorporado: arquétipos pré-definidos pra v1 (resolve a dor real — personagem inexistente — mais rápido), mas cada arquétipo permite um pequeno ajuste dentro de uma faixa (± 1 ponto entre dois dos três atributos) — meio-termo entre "travado" e "livre total", sem o custo de tela de uma distribuição livre completa.
- 3 Do dissenso do Test Engineer (#4)** — incorporado como salvaguarda adicional, não substituto: até o mecanismo de verdade estar pronto, fica reforçado (comunicação, não código) que nenhum convite novo sai — o que já era a prática em vigor, não é trabalho extra.

O plano, resumido

- Personagem:** `CharacterCreationGate` novo (Frontend Engineer, mesmo padrão visual de `LoginGate`); liga a tabela `characters` (Backend Engineer); campos e arquétipos com faixa de ajuste ± 1 (Game System Designer); texto de onboarding (Narrative Writer).
- Orçamento:** função RPC atômica no Supabase pra incrementar contador diário (Backend Engineer, atendendo o requisito de atomicidade do Code QA Engineer); `api/master.js` grava `usage.total_tokens` real de cada resposta da Groq — **confirmar na implementação se o campo existe**, não assumir (AI Master Engineer); bloqueio antes de estourar, com margem de segurança a definir.
- Teste:** fluxo de personagem entra na regra do fluxo encadeado (Test Engineer); mecanismo de simular o teto em ambiente de teste sem gastar cota real fica **em aberto** — não resolvido nesta assembleia.

- **Nenhum convite novo de teste** até as duas frentes estarem prontas e confirmadas em produção.

Em aberto — não decidido nesta assembleia

- Margem de segurança exata do bloqueio (ex.: parar em 90% do teto diário? outro número?) — decidido na implementação real, com o dado medido de verdade, não estimado.
 - Se `usage.total_tokens` realmente vem na resposta da Groq no formato usado hoje — precisa confirmação técnica direta, não suposição.
 - Como testar o bloqueio de orçamento sem gastar cota real de produção.
 - Ajuste fino (± 1) dos arquétipos — Game System Designer especifica na implementação.
-

Aguardando aprovação do Tiago

Nenhuma linha de código deste plano até o sinal explícito dele — só este documento não depende de aprovação, é processo, não produto.